Systemsoftware - Übungsblatt 1

Aufgabe 1:

Das Programm:

Ein Programm ist im eine Datei, welche im persistenten Speicher (Festplatte) eines Computers liegt. Es ist der Bauplan einer Software, zusammengesetzt aus Arbeitsanweisungen (dem Code) und statischen Daten.

Das Programm selbst macht nichts. Es verbraucht keine Ressourcen. Es ist eine reine Vorlage.

Sobald die Datei vom Betriebssystem in den flüchtigen Speicher (RAM) geladen und dort ausgeführt wird, wird aus dem Programm ein Prozess. Das bedeutet, das Programm ist die Bauvorlage für einen Prozess.

Der Prozess:

Ein Prozess ist ein Programm, dass vom Betriebssystem in den RAM geladen und gerade ausgeführt wird. Dabei hat jeder Prozess seinen eigenen, isolierten Adressraum im Speicher. Diese Isolation sorgt dafür, dass bei Absturz eines Prozesses, andere Prozesse unbeschädigt bleiben. Im Vergleich zum Programm, führt der Prozess die oben festgelegten Arbeitsanweisungen aus. Er nutzt also aktiv die Ressourcen des Rechners. Der Prozess selbst, arbeitet aber nicht. Er stellt lediglich den "Arbeitsraum" zur Verfügung. Die tatsächlichen Arbeiter innerhalb eines Prozesses sind die Threads.

Der Thread:

Ein Thread ist der Arbeiter innerhalb eines Prozesses. Er arbeitet den Code Zeile für Zeile ab und bekommt dafür die CPU als Ressource. Jeder Prozess wird mit mindestens einem Thread (Main-Thread) gestartet.

Ein Prozess kann aber beliebig viele Threads instanziieren. Die große Schwierigkeit liegt aber in der Synchronisation dieser Threads. Der große Nachteil an Multi-Threading liegt darin, dass sich Threads innerhalb eines Prozesses den Adressraum und den Heap-Speicher teilen. Wenn also mehrere Threads unkontrolliert auf die gleiche Variable zugreifen, dann entstehen Race-Conditions.

Im Großen und Ganzen bilden Programme, Prozesse und Threads also eine Hierarchie ab. Ein Programm ist die größte Einheit und steht ganz oben. Wenn ein Programm dann gestartet und ausgeführt wird, entsteht daraus ein Prozess. Der Prozess wiederrum startet einen, bis beliebig viele Threads, die dann tatsächlich die Arbeit übernehmen.

Aufgabe 2:

a) Das ein Prozess nicht ins unendliche beschleunigt werden kann, liegt an dem nicht-parallelisierbaren Anteil eines Prozesses. Wir gehen davon aus, dass ein Prozess aus einem sequentiellen Anteil und einem parallelisierbaren Anteil besteht.

Der parallelisierbare Anteil kann auf mehrere Kerne verteilt werden und (wie der Name sagt) parallel ausgeführt werden. Der sequentielle Anteil kann hingegen immer nur auf einem Kern ausgeführt werden, wie zum Problem wird. Egal wie schnell der parallelisierbare Anteil ist, der sequentielle Anteil wird zum Bottleneck, da er nicht unendlich beschleunigt werden kann. Das geht aus aus Amdahl's Gesetz hervor.

Ein Beispiel: Wir haben einen Prozess mit 20% sequentiellen und 80% parallelisierbaren Anteil. Egal wie schnell der parallelisierbare Anteil wird, der Speedup konvergiert gegen den sequenziellen (20%) Anteil.

Wir können die 80% auf n Kerne verteilen, was den parallelisierbaren Anteil auf ein Minimum schrumpft, der 20% sequenzieller Anteil bleibt trotzdem unverändert.

Im Zusammenspiel damit könnte man meinen, die Taktfrequenz des Prozessors zu erhöhen, um den sequenziellen Anteil zu beschleunigen. Dies ist prinzipiell auch korrekt, jedoch wird der Prozessor ab einer gewissen Taktfrequenz so heiß, dass die Abwärme nicht schnell genug abtransportiert werden kann. Das heißt es ist schlicht physikalisch nicht möglich, einen Prozessor unbegrenzt hochzutakten, da die Hardware sonst kaputt geht. Man kann die CPU theoretisch mit Helium oder Stickstoff kühlen, die wird dadurch jedoch so stark beschädigt, dass sie dennoch unbrauchbar ist.

=> Die Kombination aus Amdahl's Gesetz und der physikalischen Grenze durch Hitzeentwicklung sorgt also dafür, dass ein Prozess nicht unendlich weit beschleunigt werden kann.

b) Formel: $T_{MultiCore}(s,n) = T_{SingleCore}(s) + T_{SingleCore}\left(\frac{1-s}{n}\right)$

Als Basis nehmen wir einen Prozess, der auf einem Kern 1s dauert.

Parallelisierbarer Anteil: 25% Kerne: 2

$0.75 + \frac{1-0.75}{2} = 0,875$

Ausführungszeit: 0,875s

Speed up: $\frac{1}{0,875} = 1,143\times$ oder 114,3%

Parallelisierbarer Anteil: 25% Kerne: 4

$0.75 + \frac{1-0.75}{4} = 0,8125$

Ausführungszeit: 0,8125

Speed up: $\frac{1}{0,8125} = 1,23\times$ oder 123 %

Parallelisierbarer Anteil: 25% Kerne: 8

$0.75 + \frac{1-0.75}{8} = 0,78125$

Ausführungszeit: 0,78125

Speed up: $\frac{1}{0,78125} = 1,28\times$ oder 128 %

Parallelisierbarer Anteil: 25% Kerne: 16

$0.75 + \frac{1-0.75}{16} = 0,765625$

Ausführungszeit: 0,765625

Speed up: $\frac{1}{0,765625} = 1,31\times$ oder 131 %

Parallelisierbarer Anteil: 50 % Kerne: 2

$0.5 + \frac{1-0.5}{2} = 0,75$

Ausführungszeit: 0,75

Speed up: $\frac{1}{0,75} = 1,\overline{3}\times$ oder 133,3 %

Parallelisierbarer Anteil: 50 % Kerne: 4

$0.5 + \frac{1-0.5}{4} = 0,625$

Ausführungszeit: 0,625

Speed up: $\frac{1}{0,625} = 1,6\times$ oder 160 %

Parallelisierbarer Anteil: 50 % Kerne: 8

$0.5 + \frac{1-0.5}{8} = 0,5625$

Ausführungszeit: 0,5625

Speed up: $\frac{1}{0,5625} = 1,\overline{7}\times$ oder 177,7 %

Parallelisierbarer Anteil: 50 % Kerne: 16

$0.5 + \frac{1-0.5}{16} = 0,53125$

Ausführungszeit: 0,53125

Speed up: $\frac{1}{0,53125} = 1,88\times$ oder 188 %

Parallelisierbarer Anteil: 75 % Kerne: 2

$0.25 + \frac{1-0.25}{2} = 0,625$

Ausführungszeit: 0,625

Speed up: $\frac{1}{0,625} = 1,6\times$ oder 160 %

Parallelisierbarer Anteil: 75 % Kerne: 4

$0.25 + \frac{1-0.25}{4} = 0,4375$

Ausführungszeit: 0,4375

Speed up: $\frac{1}{0,4375} = 2,29\times$ oder 229 %

Parallelisierbarer Anteil: 75 % Kerne: 8

$0.25 + \frac{1-0.25}{8} = 0,34375$

Ausführungszeit: 0,34375

Speed up: $\frac{1}{0,34375} = 2,\overline{90}\times$ oder 290 %

Parallelisierbarer Anteil: 75 % Kerne: 16

$0.25 + \frac{1-0.25}{16} = 0,296875$

Ausführungszeit: 0,296875

Speed up: $\frac{1}{0,296875} = 3,37\times$ oder 337 %

Nr. 3.)

Wie zu erwarten läuft das Programm schneller, je mehr Threads genutzt wird. Die von mir genutzte Datei enthält ~3.500 Zeilen. Das Programm mit einem Thread benötigt circa 33ms. Bereits bei der Nutzung von zwei Threads verkürzt sich die Dauer auf 25ms. Aufgrund der geringen Anzahl an Zeilen, verkürzt sich die Zeit mit mehr Threads nicht mennenswert weiter. Wenn man an der Maximum geht (z.B. 32.000 Threads) verlängert sich die Zeit auf circa 120ms. Ab dem 32768-ten Thread wird eine Exception geworfen, da der Pool nur 32767 Threads erzeugen kann (Laut Java Doku).

Zusammenfassend können wir den zu erwartenen Effekt erkennen. Auch das die Zeit ab Thread x länger wird, entspricht den Erwartungen.